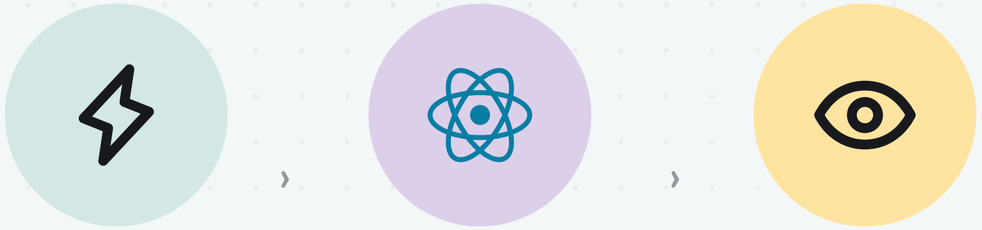
React state *is memory with a queue*

- Cover
- Snapshot
- Declare
- Queue
- Batching
- Rules
- One render
- Close

State and setState: updates, batching, re-render behaviour



Event

a click, a tick, a reply

State

the value between renders

Render

the UI reads the snapshot

State is a snapshot, and the setter queues the next one

Three facts. Each one decides how the UI moves.

- Cover
- Snapshot
- Declare
- Queue
- Batching
- Rules
- One render
- Close

SNAPSHOT

One render, one value

The handler reads the value from its own render, never the next one.

Key point: count + 1 uses the count of this render

QUEUE

setState asks for a render

The setter queues the next render. React runs it after the handler.

Key point: the screen updates after the queue drains

SCOPE

State belongs to one component

Each mounted component owns its own value. Nothing leaks across.

Key point: two counters never share one count

The value is fixed per render. The setter plans the next one.

useState declares the memory one component owns

One call, one value, one setter. Nothing is shared.

- Cover
- Snapshot
- Declare
- Queue
- Batching
- Rules
- One render
- Close

JAVASCRIPT — 6 lines, 24px

```
1  const [count, setCount] = useState(0)
2  function bump() {
3    setCount(count + 1) // same snapshot
4    setCount(count + 1) // same snapshot
5    setCount(count + 1) // still plus one
6  }
```

Three calls with one snapshot still add one.

The updater form reads the queue, not the snapshot

Same three calls. This time the count moves by three.

- Cover
- Snapshot
- Declare
- Queue
- Batching
- Rules
- One render
- Close

JAVASCRIPT

6 lines, 24px

```
1  const [count, setCount] = useState(0)
2  function bump() {
3    setCount(c => c + 1) // queue one
4    setCount(c => c + 1) // queue two
5    setCount(c => c + 1) // count plus three
6  }
```

Read the queue with c => c + 1 and three calls add three.

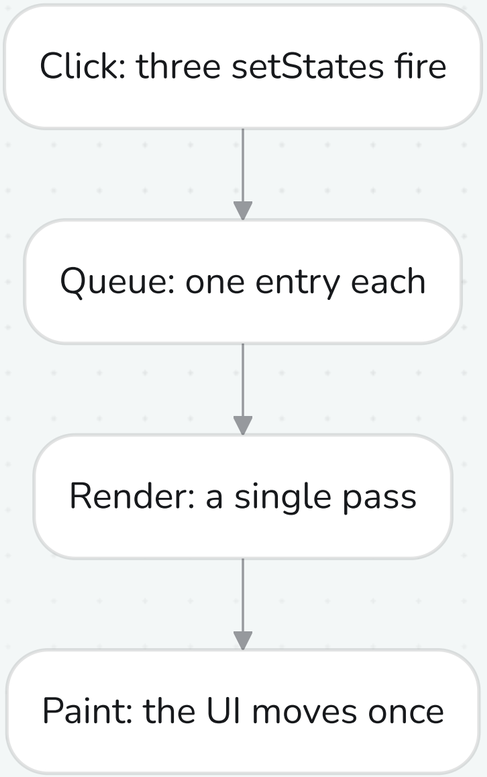
Cover
Snapshot
Declare
Queue
Batching
Rules
One render
Close

When do three updates become one render?

Batching turns one event into one render

The queue collects the calls. The render runs once.

- Cover
- Snapshot
- Declare
- Queue
- Batching
- Rules
- One render
- Close



THE QUEUE

1

Render for three calls in one handler.

THE COST

0

Renders when state is mutated in place.

THE ORDER

3

Queued updates applied in order.

One handler means one render, however many calls it holds.

Three rules keep state predictable

Each rule carries the test that proves it.

- Cover
- Snapshot
- Declare
- Queue
- Batching
- Rules
- One render
- Close

Replace, never mutate

Write a new object or array. A push moves nothing on screen.

Exit test: the old value is untouched

Queue with the updater form

Pass c => c + 1 when the next value needs the last one.

Exit test: three calls move the count by three

Lift only what is shared

Keep state next to its reader. Share it only when two readers need it.

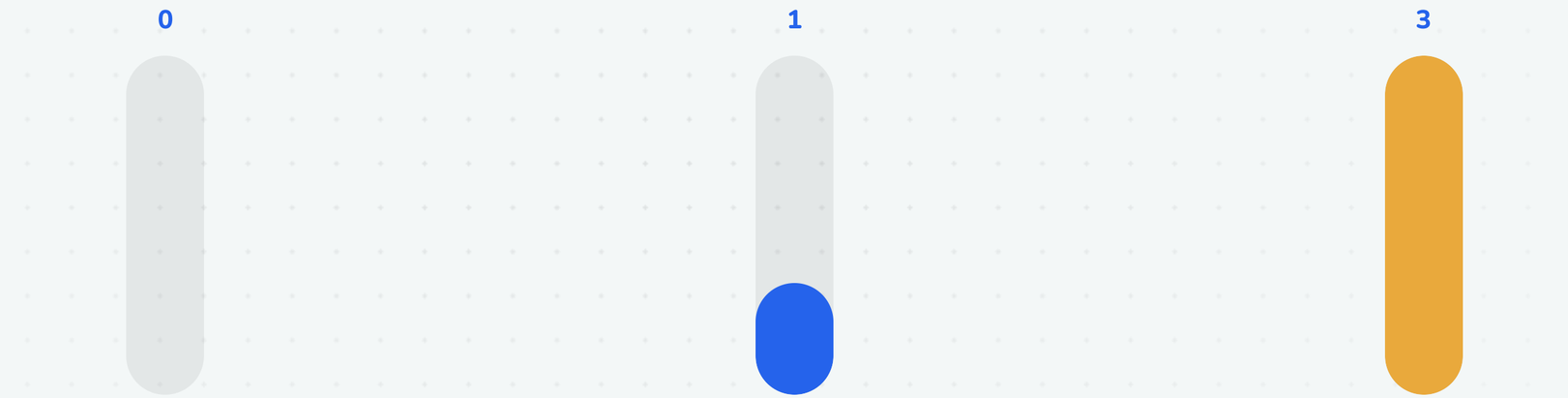
Exit test: one change renders one subtree

New values, queued updates, close owners. Nothing else renders.

Three calls in one handler cost one render

The bar is the render count. The label names the pattern.

- Cover
- Snapshot
- Declare
- Queue
- Batching
- Rules
- **One render**
- Close



demo counts, React 18 · one click each · the queue decides, not the calls

Batching spends one render per event, never one per call.

The line worth repeating after the meeting

- Cover
- Snapshot
- Declare
- Queue
- Batching
- Rules
- One render
- Close

State is a snapshot.
The setter is a queue.
The render is the receipt.

React states: state, setState, batching, re-render behaviour

1
render for three calls in one handler

0
renders when state is mutated

3
rules that keep state predictable

A teaching deck. The counts are demo counts, stated on the slide that shows them.